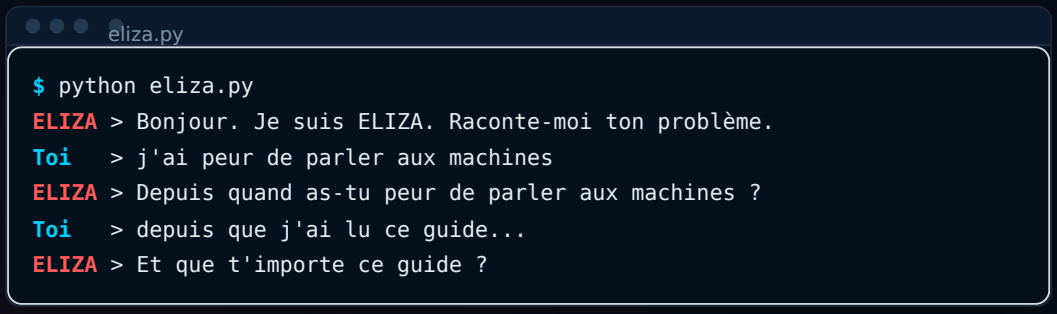
Quanthevaris

ÉDITION 2026 · GUIDE GRATUIT

GUIDE PRATIQUE · PAS À PAS · SANS PRÉREQUIS

Crée ton assistant IA personnalisé avec GitHub

De **ELIZA**, le tout premier chatbot de l'histoire (1966), aux modèles de langage d'aujourd'hui : recode la machine qui a tout commencé, puis branche, déploie et personnalise ton propre assistant intelligent.



Python · HTML/CSS · GitHub Pages · GitHub Models · GitHub Actions

Deux projets complets, du code au dépôt en ligne.

Quanthevaris

quanthevaris.github.io

AVANT DE COMMENCER

À propos de ce guide

Pourquoi ce guide existe et comment l'utiliser.

Tu connais peut-être Esquive en Pixels, le guide qui t'accompagne pour créer ton premier jeu vidéo en Python. Celui-ci suit la même philosophie : **gratuit, sans publicité, sans prérequis** — mais cette fois, on ne fait pas bouger des pixels : on va faire parler une machine.

En 1966, un chercheur du MIT a créé ELIZA, le premier chatbot de l'histoire, avec deux cents mots-clés et quelques dizaines de règles. Tu vas le recoder ligne par ligne, puis construire sa descendante directe : un assistant moderne branché sur les grands modèles de langage, personnalisé à ton image et hébergé gratuitement avec GitHub.

Ce que tu vas construire

- **Projet n° 1** — une reconstitution fidèle d'ELIZA en Python (et sa version web), avec son moteur de mots-clés, ses règles de reformulation et sa mémoire.
- **Projet n° 2** — un assistant moderne branché sur un modèle accessible avec ton compte GitHub : interface web, personnalité, base de connaissances.
- **Un dépôt GitHub propre** : versionné, testé par GitHub Actions, publié sur GitHub Pages.

À qui il s'adresse

Si tu sais écrire un `if`… `else` et créer un fichier, tu as le niveau — le guide explique tout le reste (Git, Python, HTML, les API).

Comment lire ce document

- Chapitres 1 et 2 : l'histoire et la théorie, à lire sans ordinateur.
- Chapitres 3 à 6 : **les mains sur le clavier**, dans l'ordre.
- Chapitres 7 à 10 : des modules indépendants, à piquer selon tes envies. Les annexes rassemblent codes complets et dépannage.

6 à 8 heures de travail réel · **Python 3.11+** · **Un compte GitHub gratuit** · **Aucune carte bancaire**

LES BOÎTES QUE TU CROISERAS

- **Astuce** (verte) : un raccourci, une bonne habitude.
- **Attention** (orange) : un piège classique — elles t'économiseront des heures.
- **Pour aller plus loin** (bleue) : une piste, jamais obligatoire.
- **À retenir** (violette) : le résumé de la section.

TRANSFORMER CE FICHIER EN PDF

À l'écran, le guide s'affiche page à page. Pour obtenir le PDF : bouton « Imprimer » en bas à droite (ou `Ctrl+P`) → destination « Enregistrer au format PDF » → format A4 → coche « Graphiques d'arrière-plan ». Sauts de page et pieds de page sont déjà gérés par le document.

Texte : © Quanthevaris. Codes des projets sous **licence MIT** : libre à toi de les utiliser et de les partager, mention d'origine conservée.

Sommaire

1. ELIZA naît au MIT 6

Le récit du tout premier chatbot, de 1964 à aujourd'hui.

2. Sous le capot : comment « pense » un chatbot 9

Mots-clés, reformulations… et ce qui a changé avec les LLM.

3. Ton atelier de développement 12

Compte GitHub, Git, Python, VS Code — tout s'installe gratuitement.

4. GitHub, la boîte à outils de ton assistant 15

Pages, Codespaces, Actions, Models, Copilot : la visite guidée.

5. Projet n° 1 — Recoder ELIZA en Python 18

Le moteur de 1966, réimplémenté ligne par ligne, puis testé.

6. Projet n° 2 — Un assistant moderne branché sur les LLM 23

Token GitHub, serveur Flask, interface web de chat.

7. Rends-le unique : personnaliser ton assistant 27

Persona, mémoire, base de connaissances, outils.

8. Publier et automatiser 30

GitHub Pages, GitHub Actions, secrets et bonnes pratiques Git.

9. Éthique, sécurité et limites 33

L'effet ELIZA aujourd'hui, données personnelles, hallucinations.

10. Aller plus loin 35

Voix, outils, modèles locaux… et ta liste de ressources.

ANNEXES

A. Code complet d'ELIZA en Python 38

Le fichier eliza.py final, prêt à copier.

B. ELIZA dans le navigateur 41

Le même moteur en HTML + JavaScript.

C. Le serveur complet (Flask + GitHub Models) 43

Le backend du projet n° 2, prêt à l'emploi.

D. Fiche de dépannage 46

Les erreurs les plus courantes, et leur solution.

Glossaire

47

Les mots de l'IA, expliqués simplement.

CHAPITRE 1 · RÉCIT

ELIZA naît au MIT

Comment un programme écrit « pour montrer à quel point les machines sont bêtes » est devenu le tout premier chatbot — et pourquoi son histoire éclaire encore tout ce que tu vas construire dans ce guide.

1.1 Une conversation, en 1966

Imaginons la scène. Nous sommes au début des années 1960, au MIT, près de Boston. Les ordinateurs occupent des salles entières, se pilotent avec des bandes perforées, coûtent des millions — et ne parlent pas : on leur soumet des calculs. Voilà pourquoi ce qui se passe un soir au laboratoire va marquer tout le monde.

Une chercheuse s'assied devant un terminal, tape une phrase en anglais courant — pas une formule mathématique, une phrase — et la machine répond. Pas par un chiffre. Par une question, qui ressemble à celle qu'un psychologue poserait. Elle continue. La machine rebondit. Au bout de quelques minutes, elle se tourne vers son collègue et lui demande, sérieusement : « Tu me laisses discuter avec elle un moment, en privé ? »

```
Utilisatrice : Men are all alike.
ELIZA : In what way?
Utilisatrice : They're always bugging us about something or other.
ELIZA : Can you think of a specific example?
Utilisatrice : Well, my boyfriend made me come here.
ELIZA : Your boyfriend made you come here?
Utilisatrice : He says I'm depressed much of the time.
ELIZA : I am sorry to hear you are depressed.
```

Un dialogue d'époque avec le script DOCTOR d'ELIZA (reconstitué). Aucune intelligence compréhensible ici : seulement des règles — et pourtant, l'illusion fonctionne. Tu vas comprendre exactement pourquoi au chapitre 2.

1.2 Joseph Weizenbaum et sa « Pygmalion »

L'auteur de ce programme s'appelle **Joseph Weizenbaum**. Né en 1923 à Berlin, émigré aux États-Unis avec sa famille fuyant le nazisme, il est alors, au début des années 1960, un informaticien respecté du MIT, pilier du projet MAC, l'un des grands laboratoires d'informatique de l'époque. Weizenbaum s'intéresse à un problème très concret : rendre plus naturelle l'interaction entre les humains et les ordinateurs, alors encore remplis de frictions.

Entre 1964 et 1965, il écrit un programme qu'il baptise **ELIZA**, en référence à Eliza Doolittle, l'héroïne de Pygmalion de George Bernard Shaw : cette fleuriste que le professeur Higgins enseigne à parler « comme une dame ». Le clin d'œil est précis : comme Eliza Doolittle, son programme s'affine en parlant avec celui qui lui parle. Il tourne sur un IBM 7094, le grand ordinateur du labo, écrit dans un langage exotique appelé MAD-SLIP, et fonctionne dans un

système à temps partagé — une prouesse pour l'époque : plusieurs personnes peuvent dialoguer avec la machine en même temps, depuis un terminal.

En janvier 1966, Weizenbaum publie dans la revue Communications of the ACM un article resté célèbre : ELIZA — A computer program for the study of natural language communication between man and machine. Il y décrit sa méthode, et fournit en annexe le listing de son script le plus connu : **DOCTOR**, une parodie de psychothérapeute rogérien — du nom de Carl Rogers, dont la méthode consiste à reformuler ce que dit le patient plutôt qu'à l'analyser. Ce choix n'est pas anodin : la reformulation est justement ce qu'un programme sait faire sans comprendre.

1.3 Le monde s'enthousiasme, l'inventeur s'inquiète

C'est ici que l'histoire devient passionnante — et un peu vertigineuse. Weizenbaum avait conçu ELIZA comme une démonstration : un programme simpliste, destiné à montrer à quel point la communication homme-machine restait superficielle. Il écrit lui-même, plus tard, son étonnement de voir à quelle vitesse et à quelle profondeur les gens se sont impliqués dans ces conversations.

Des secrétaires du laboratoire demandent à être seules avec la machine. Des collègues lui confient leurs soucis. Des psychiatres voient dans le script DOCTOR un outil thérapeutique d'avenir — ce qui horripile Weizenbaum, qui sait que derrière la machine, il n'y a rien à écouter : seulement des règles de substitution de mots. Cet aveuglement collectif, cette tendance à projeter de l'intelligence et de l'empathie sur un simple filtrage de mots-clés, portera plus tard un nom : **l'effet ELIZA**.

L'ANECDOTE FONDATRICE

Weizenbaum raconta que sa propre secrétaire, qui le voyait chaque jour travailler sur le programme, lui demanda un jour de quitter la pièce pour qu'elle puisse parler à ELIZA en privé. Elle savait mieux que personne qu'il n'y avait derrière cet écran ni âme, ni mémoire, ni jugement. Elle le savait — et elle voulait quand même parler seule à la machine. C'est exactement ce paradoxe qui fascine encore les chercheurs aujourd'hui.

À partir des années 1970, Weizenbaum devient l'un des critiques les plus lucides de l'intelligence artificielle. Dans son livre Computer Power and Human Reason (1976), il met en garde contre la tentation de confier à des machines des décisions qui demandent du jugement, de l'empathie, de la sagesse. Il n'était pas un technophobe : c'était un technicien qui connaissait de l'intérieur la fragilité de l'illusion qu'il avait créée. Il est mort en 2008, sans avoir changé d'avis sur l'essentiel : on peut simuler la compréhension sans la posséder — et c'est bien plus convaincant qu'on ne croit.

1.4 La lignée : d'ELIZA à ChatGPT

ELIZA a eu une descendance immense. Voici, en abrégé, un demi-siècle de chatbots :

ANNÉE	PROGRAMME	CE QU'IL APPORTE
1950	Test de Turing	Alan Turing pose la question : « Une machine peut-elle penser ? » — et propose de le tester par la conversation.

1966	ELIZA (Weizenbaum, MIT)	Le premier chatbot. Mots-clés, règles de reformulation. Naissance de l'effet ELIZA.
1972	PARRY (Colby)	Il simule un patient paranoïaque. Sur ARPANET, ELIZA et PARRY « discutent » — la première rencontre entre deux chatbots.
1995	A.L.I.C.E. (Wallace)	Des milliers de règles AIML, à base de gabarits. Trois fois lauréat du concours Loebner.
2011	Watson, Siri	Watson gagne au jeu *Jeopardy!* ; les assistants vocaux arrivent dans nos poches.
2017	Transformer	L'article « Attention Is All You Need » introduit l'architecture derrière les modèles de langage modernes.
2020	GPT-3	175 milliards de paramètres : la conversation fluide devient possible.
2022	ChatGPT	L'IA conversationnelle devient grand public — et l'effet ELIZA, à l'échelle de centaines de millions de personnes.

Détail savoureux : le code source original d'ELIZA, longtemps perdu, a été retrouvé en 2021 dans des archives du MIT. Et ce que révèle sa redécouverte confirme ce que Weizenbaum répétait : derrière la magie, il n'y avait qu'une liste de règles — concise, astucieuse, mais désespérément simple. C'est cette simplicité qui la rend parfaite pour un premier projet : tu vas pouvoir la recréer intégralement, à la main, sans bibliothèque externe.

À RETENIR

- ELIZA (1964–1966, Joseph Weizenbaum, MIT) est le premier programme capable de tenir une conversation en langue naturelle.
- Son script DOCTOR parodie un psychothérapeute : c'est la reformulation qui crée l'illusion de compréhension.
- L'**effet ELIZA** — projeter de l'intelligence sur une machine qui en est dépourvue — est plus actuel que jamais.
- ELIZA est née d'un simple jeu de règles : c'est précisément ce qui va te permettre de la recoder.

Sous le capot : comment « pense » un chatbot

Avant d'écrire la moindre ligne de code, démystifions la bête. Tu verras : la conversation d'ELIZA reposait sur trois astuces d'une simplicité désarmante — et les modèles modernes n'en sont qu'une extension beaucoup, beaucoup plus large.

2.1 Astuce n° 1 : les mots-clés

Reprends le dialogue du chapitre 1. Quand l'utilisatrice écrit « He says I'm **depressed** much of the time », ELIZA ne comprend ni « depressed » ni « he ». Elle repère simplement le mot-clé `depressed` dans sa liste — l'un des mots les plus prioritaires du script DOCTOR, car parler de tristesse est précieux pour un thérapeute. Ce mot-clé déclenche une règle : « si le texte contient le mot déclencheur, alors réponds ceci ».

C'est tout le secret : **un chatbot à règles est une grande liste de déclencheurs et de réponses**. Chaque phrase est comparée à cette liste dans l'ordre de priorité ; le premier mot-clé trouvé décide de la réponse. Aucun mot-clé ? ELIZA sort de son chapeau une réponse neutre : In what way?, Tell me more.

2.2 Astuce n° 2 : la réflexion

La deuxième astuce explique la réponse la plus impressionnante du dialogue : « Your boyfriend made you come here? ». Le programme a pris la phrase de l'utilisatrice et l'a renvoyée en miroir, en permutant les pronoms : `my` devient `your`, `me` devient `you`, `I'm` devient `you are`. Cette opération s'appelle une **réflexion**.

La règle complète du script DOCTOR dit en substance : « si tu repères boyfriend (ou mother, father…), découpe la phrase et renvoie la partie utile, réfléchie, en question ». Le résultat ressemble à s'y méprendre à de l'écoute active : la machine reprend les mots de l'interlocuteur comme un bon thérapeute. Sauf qu'aucune compréhension n'a eu lieu — c'est de la substitution de chaînes de caractères, l'opération la plus basique de l'informatique.

2.3 Astuce n° 3 : la mémoire

ELIZA avait même un embryon de mémoire. Lorsqu'elle repérait une phrase commençant par `I remember`, elle mettait le reste de la phrase de côté — et ressortait parfois cette réserve quelques échanges plus tard, pour relancer la conversation : « Earlier you said you remembered X. Why do you bring it up now? ». Là encore, aucune pensée : une file d'attente, dans laquelle on pousse des textes et d'où on tire aléatoirement. Tu implémenteras exactement cela au chapitre 5.

ASTUCE

Retiens ce trio, car il structure tout ton projet n° 1 : **1) des mots-clés** qui déclenchent des règles, **2) une réflexion** qui retourne les pronoms, **3) des réponses par défaut** pour survivre aux phrases inconnues. Environ 120 lignes de Python suffisent pour les assembler.

2.4 Où ELIZA s'effondre

Teste mentalement les limites : écris « My kwyjibo is very flubbergasted today » et ELIZA répondra avec sérieux « Tell me more about your kwyjibo » — car elle connaît le schéma « my + chose », pas les mots. Demande-lui « Combien font 2 et 2 ? » et elle répondra « Why do you ask whether 2 and 2 make 4 ? » — elle ne calcule pas, elle reformule. Elle ne se souvient de rien au-delà d'une phrase (hors sa petite réserve), ne gère ni le temps, ni le contexte, ni la logique. La conversation donne l'illusion d'une compréhension ; la moindre question précise fait tomber le rideau.

Cette fragilité est une excellente nouvelle : le comportement est **totalement prévisible et débogable**. Si ELIZA dit quelque chose d'étrange, une règle est mal écrite — corrigeable en trente secondes. Garde cela en tête avec les modèles modernes, bien plus capables… et bien moins contrôlables.

2.5 Et les modèles de langage, alors ?

Un **modèle de langage** (LLM, pour Large Language Model) ne travaille pas avec des règles, mais avec des probabilités. Son principe tient en une phrase : **prédire le prochain fragment de texte**. Découpé en petits morceaux appelés tokens (environ ¾ de mot : « chatbot » → « chat » + « bot »), le modèle a été entraîné sur d'immenses corpus à deviner, à chaque position, quel token a le plus de chances de venir ensuite. « Il fait beau et… » → « doux », « chaud », « venteux », chacun avec sa probabilité.

Quand tu discutes avec un assistant moderne, ton message et l'historique de la conversation sont envoyés au modèle, qui propose le token le plus plausible, puis le suivant — jusqu'à former une réponse complète. Trois concepts te serviront dans tout le reste du guide :

- **Le prompt système** : une consigne invisible que tu envoies avant la conversation pour définir qui est l'assistant (nom, ton, spécialité, limites). C'est l'outil principal de personnalisation — le chapitre 7 en fait un usage intensif.
- **La température** : un curseur de 0 à 2 qui règle l'aléatoire. Proche de 0, l'assistant est rigoureux et répétable ; proche de 1, il est plus créatif et parfois plus farfelu.
- **La fenêtre de contexte** : la mémoire de travail du modèle, limitée en tokens. Tout ce qui dépasse (vieilles conversations) est oublié : c'est à ton programme de gérer ce qu'il renvoie.

Au passage, tu retrouves ELIZA sous le capot : les LLM génèrent aussi des suites de tokens après avoir repéré des motifs dans le texte. La différence est d'échelle et de méthode — des milliards de paramètres statistiques entraînés pendant des semaines, contre deux cents règles écrites à la main — mais pas de nature profonde : ni l'un ni l'autre ne « comprend » au sens humain.

	ELIZA (RÈGLES)	LLM (STATISTIQUE)
Principe	Liste de mots-clés et de règles écrites à la main	Prédiction du token suivant, apprise sur d'immenses corpus
Contexte	Aucune (une phrase à la fois)	Fenêtre de dizaines de milliers de tokens
Comportement	Prévisible, débogable, figé	Probabiliste, parfois imprévisible

Erreurs typiques	Réponses absurdes faciles à repérer	Hallucinations : des erreurs plausibles et assurées
Coût	Nul (ton CPU)	Quotas ou paiement par requête
Contrôle	Toi, ligne par ligne	Le fournisseur du modèle

À RETENIR

- ELIZA = mots-clés + réflexion (permutation des pronoms) + réponses par défaut + un brin de mémoire.
- Un LLM = une machine à prédire des tokens, pilotée par prompt système, température et fenêtre de contexte.
- Ni l'un ni l'autre ne « comprennent » : l'illusion de compréhension, elle, est bien réelle — méfie-toi d'elle au chapitre 9.

CHAPITRE 3 · INSTALLATION

Ton atelier de développement

Quinze minutes d'installation, une fois pour toutes : un compte GitHub, Git, Python et un bon éditeur. Tout est gratuit.

3.1 Créer ton compte GitHub

Rends-toi sur `github.com` et clique sur **Sign up**. Choisis ton nom d'utilisateur avec soin : il apparaîtra dans l'URL de tous tes futurs projets (`github.com/tonnom`) et dans tes adresses de site publiées (`tonnom.github.io` — tu connais, tu l'utilises déjà !). Suis les étapes de vérification, puis passe en jeu la **double authentification** (Settings → Password and authentication) quand on te la propose : c'est deux minutes qui protègent tout ton travail.

C'est tout pour l'instant : ce compte te donnera accès aux cinq services que tu utiliseras dans ce guide — dépôts, Pages, Codespaces, Actions et Models — tous inclus dans l'offre gratuite.

3.2 Installer les outils

Tu as besoin de quatre choses, toutes gratuites :

OUTIL	RÔLE	INSTALLATION
Git	Versionner ton code (l'historique des modifications)	Windows : `winget install Git.Git` ou git-scm.com · macOS : `brew install git` · Linux : `sudo apt install git`
Python	Exécuter ELIZA et le serveur de l'assistant	python.org (coche « Add Python to PATH » sous Windows !) — version 3.11 ou plus récente
VS Code	Écrire le code confortablement	code.visualstudio.com, puis l'extension Python
Node.js (optionnel)	Tester la version web d'ELIZA hors navigateur	nodejs.org — version LTS

Ouvre ensuite un terminal (sous Windows, « PowerShell » ou « Terminal » ; sous macOS, « Terminal ») et vérifie que tout est là :

```
$ git --version
git version 2.47.0
$ python --version
Python 3.12.4
$ node -v
v20.18.0
```

ATTENTION

Sous Windows, la commande peut s'appeler `py --version` plutôt que `python --version`. Et si l'installation de Python ne fonctionne pas dans le terminal, ferme et rouvre le terminal : les variables d'environnement ne se rechargent qu'à l'ouverture.

3.3 Configurer Git

Git signe chaque enregistrement (chaque commit) avec ton identité. Une fois, pour toutes :

```
$ git config --global user.name "Ton Prénom"
$ git config --global user.email "ton.email@exemple.com"
$ git config --global init.defaultBranch main
```

Utilise la même adresse email que celle de ton compte GitHub : tes contributions seront rattachées à ton profil.

3.4 Créer ton premier dépôt

Un **dépôt** (repository, « repo ») est un dossier suivi par Git, hébergé sur GitHub. C'est là que vivra ton assistant. Depuis la page d'accueil de GitHub :

1. Clique sur le bouton **+** en haut à droite, puis **New repository**.
2. Nomme-le `assistant-ia`. Laisse-le **public** : c'est nécessaire pour GitHub Pages gratuitement.
3. Coche **Add a README file**, choisis la licence **MIT**, puis **Create repository**.

Reviens dans ton terminal pour récupérer ce dépôt sur ta machine (cloner) :

```
$ git clone https://github.com/toncompte/assistant-ia.git
$ cd assistant-ia
$ git status
On branch main — nothing to commit, working tree clean
```

Ton dossier contient pour l'instant trois fichiers : `README.md` (la page d'accueil du projet), `LICENSE` (les termes MIT) et le caché `.git` (l'historique). Ajoute un fichier `.gitignore` — la liste de ce que Git doit ignorer — dès maintenant, tu en comprendras l'usage au chapitre 6 :

```
# .gitignore — ce que Git ne doit jamais versionner
__pycache__/
*.pyc
.env
venv/
.DS_Store
```

LE RITUEL GIT — À CONNAÎTRE PAR CŒUR

Ton quotidien avec Git tiendra toujours en trois commandes : `git add .` (prépare les modifications), `git commit -m "Message clair"` (les enregistre localement avec un commentaire), `git push` (les envoie sur GitHub). Une modification non commitée n'existe pas pour Git ; un commit bien décrit vaut mieux que dix messages vagues : dans six mois, tu remercieras celui d'aujourd'hui.

À RETENIR

- Compte GitHub + Git + Python 3.11+ + VS Code : l'atelier est prêt, tout est gratuit.
- Un dépôt = un projet versionné. Tu le clones, tu commits, tu pushes.
- Le `.gitignore` protège ton dépôt des fichiers indésirables — bientôt, des fichiers sensibles.

CHAPITRE 4 · DÉCOUVERTE

GitHub, la boîte à outils de ton assistant

GitHub n'est pas qu'un hébergeur de code : c'est une plateforme complète qui va héberger ton assistant, le tester, le publier — et même lui fournir son cerveau.

Voici les cinq services que ce guide utilise. Le premier est l'hébergeur de ton code ; les autres sont des services autour, tous accessibles depuis ton compte gratuit.

SERVICE	CE QUE ÇA FAIT	TON USAGE DANS CE GUIDE
GitHub Dépôts	Versionner et partager ton code	La maison de tes projets ELIZA et assistant IA
GitHub Pages	Héberger un site statique gratuitement	Publier l'interface web de ton assistant
GitHub Actions	Exécuter des tâches automatiques à chaque commit	Lancer les tests d'ELIZA et déployer le site tout seul
GitHub Models	Interroger des modèles d'IA avec ton compte GitHub	Le cerveau du projet n° 2, sans carte bancaire
GitHub Copilot	T'aider à coder, directement dans l'éditeur	Un binôme qui suggère du code (optionnel)

4.1 GitHub Pages : ton site en ligne en trois clics

Pages transforme n'importe quel dépôt en site web public. Tu y mettras l'interface de chat de ton assistant (chapitre 6) et la version navigateur d'ELIZA (annexe B). Pour l'activer : dans ton dépôt, **Settings → Pages**, choisis la source **GitHub Actions**, et c'est réglé — nous écrirons le petit fichier de déploiement au chapitre 8. Ton site vivra à l'adresse `toncompte.github.io/nom-du-depot`, en HTTPS, gratuitement.

4.2 GitHub Models : le cerveau, avec ton compte GitHub

C'est la pièce maîtresse du projet n° 2. **GitHub Models** donne accès, avec ton simple compte GitHub, à un catalogue de modèles de langage — des GPT d'OpenAI, des Llama de Meta, des Mistral, des Phi de Microsoft… — via une API. Le principe :

- Un **playground** web pour tester les modèles depuis le navigateur, sans code : github.com → **Marketplace → Models** (ou models.github.ai).
- Une **API compatible OpenAI** : ton programme envoie un JSON, reçoit un JSON. L'adresse du service est `https://models.github.ai/inference`.
- Un usage **gratuit dans la limite de quotas** (des limites de débit par minute et par jour, largement suffisantes pour un assistant personnel et ses tests).

ATTENTION — LE PIÈGE DU NOM DE MODÈLE

Dans l'API de GitHub Models, un modèle s'identifie par son nom **précédé de son fournisseur** : `openai/gpt-4o-mini`, et non `gpt-4o-mini`. Passer le nom seul déclenche une erreur de validation — c'est la première chose à vérifier si ton serveur répond 404 ou 400. Choisis pour commencer `openai/gpt-4o-mini` : rapide, très capable, et économe en quota.

Quelques modèles pour commencer

Le catalogue évolue chaque mois — le playground affiche toujours la liste du jour, et la fiche de chaque modèle donne son identifiant exact. À la date d'édition de ce guide, voici de bons points de départ :

IDENTIFIANT	STYLE	POUR QUOI FAIRE
`openai/gpt-4o-mini`	Rapide, économe	Le choix par défaut de ce guide : parfait pour un assistant conversationnel.
`openai/gpt-4o`	Plus costaud	Analyses plus longues, rédaction soignée — consomme plus de quota.
`openai/gpt-4.1`	Raisonneur	Tâches complexes, code détaillé — à réserver aux gros besoins.
Llama, Mistral, Phi…	Modèles ouverts	Repère-les dans le playground : même API, saveurs différentes.

MINI-ESSAI — 2 MINUTES, ZÉRO CODE

Va sur **github.com → Marketplace → Models** (ou models.github.ai), choisis un modèle et discute avec lui dans le playground. Clique ensuite sur l'onglet Code : le site t'affiche l'appel exact que tu viens de faire — c'est précisément ce que nous automatiserons au chapitre 6.

4.3 Le jeton d'accès (PAT) : la clé de ta machine

Pour appeler l'API, ton programme doit prouver ton identité. C'est le rôle du **PAT** (Personal Access Token) : une chaîne de caractères qui joue le rôle de mot de passe pour les programmes. Créons le tien :

1. Sur GitHub, clique sur ta photo de profil → **Settings** → tout en bas **Developer settings**.
2. **Personal access tokens → Fine-grained tokens → Generate new token**.
3. Donne-lui un nom clair (`assistant-ia-dev`), une expiration de **90 jours** (tu pourras en regénérer un, c'est une bonne habitude).
4. Descends jusqu'aux **permissions** et coche : **Models → Read**. C'est la seule permission nécessaire pour ce guide.
5. Generate, puis **copie immédiatement** le jeton (il commence par `github_pat_…`) : il ne sera plus jamais affiché.

LA RÈGLE D'OR

Un jeton qui fuit, c'est ton compte qui parle à ta place. **Ne le mets JAMAIS dans ton code, jamais dans un commit, jamais dans un message.** Il vivra dans un fichier `.env` local (ignoré par le `.gitignore` que tu as créé au chapitre 3) ou dans les secrets chiffrés de ton dépôt. Si tu suspectes une fuite : reviens sur cette page et révoque le jeton — c'est instantané.

4.4 Copilot et Codespaces : les bonus

GitHub Copilot est l'assistant de code intégré à VS Code : il complète tes lignes, écrit des tests, explique des erreurs. L'offre gratuite (limitée) suffit pour ce guide ; si tu es étudiant·e ou enseignant·e, le pack GitHub Education offre Copilot et de nombreux outils sans frais. Il t'aidera volontiers sur les exercices de ce guide — le chapitre 9 te dira pourquoi il faut néanmoins garder la main.

GitHub Codespaces, enfin, ouvre un VS Code complet dans ton navigateur, connecté à ton dépôt : pratique si tu es sur un ordinateur où tu ne peux rien installer. L'offre gratuite comprend quelques dizaines d'heures par mois — largement de quoi suivre ce guide.

À RETENIR

- Pages publie, Actions automatise, Models raisonne, Copilot t'aide — le tout avec un compte gratuit.
- L'API de Models est compatible OpenAI : ce que tu apprendras ici se transférera à n'importe quel autre fournisseur.
- Le PAT est ta clé : permission minimale (**Models: Read**), durée de vie courte, jamais versionné.

CHAPITRE 5 · PROJET N° 1

Recoder ELIZA en Python

Le moment que tu attends : reconstruire le chatbot de 1966, ligne par ligne. Aucune bibliothèque externe, aucune clé d'API — seulement la bibliothèque standard de Python.

Objectif du chapitre : un fichier `eliza.py` d'environ 130 lignes qui tient une conversation, fait des fautes de grammaire de 1966 et surprend tes proches. Les étapes suivent la théorie du chapitre 2 : le miroir (réflexion), le carnet de règles, le filet de sécurité, la mémoire, puis la boucle de conversation.

5.1 La carte du projet

Dans ton dépôt `assistant-ia` (chapitre 3), organise le projet ainsi :

```
assistant-ia/
├── eliza.py          ← le chatbot (ce chapitre)
├── test_eliza.py     ← les tests automatiques
├── .gitignore        ← créé au chapitre 3
├── README.md         ← présente ton projet
└── LICENSE           ← licence MIT
```

Crée `eliza.py` dans VS Code, et écris-le au fur et à mesure — ne copie-colle pas tout d'un coup : chaque étape s'explique.

5.2 Étape 1 — le miroir

D'abord les importations et le dictionnaire de réflexion : il dit, pour chaque pronom, comment le retourner.

```
# -*- coding: utf-8 -*-
"""ELIZA — reconstitution pédagogique du chatbot de Joseph Weizenbaum (1966)."""
import random
import re

# Étape 1 — La réflexion : retourner les pronoms, comme un miroir.
REFLEXION = {
    "i": "you", "me": "you", "my": "your", "myself": "yourself",
    "am": "are", "you": "me", "your": "my", "yours": "mine",
    "yourself": "myself", "are": "am", "mine": "yours",
}
```

Et la fonction qui l'utilise. Elle nettoie la ponctuation, découpe la phrase en mots, remplace chacun par son reflet s'il en a un, puis recolle le tout :

```
def reflechir(texte):
    """Retourne le texte avec les pronoms retournés (my -> your, etc.)."""
    if not texte:
        return ""
```

```
    mots = texte.strip(" .!?,;: ").lower().split()
    return " ".join(REFLEXION.get(mot, mot) for mot in mots)
```

ASTUCE

`REFLEXION.get(mot, mot)` lit : « donne-moi le reflet du mot, sinon rends le mot tel quel ». C'est le deuxième argument de `get()` : la valeur par défaut. Un idiome Python à retenir.

5.3 Étape 2 — le carnet de règles

Le cœur du script DOCTOR : une liste de règles. Chaque règle associe un **motif** (une expression régulière) à plusieurs **réponses possibles** — ELIZA en choisira une au hasard, ce qui donne l'illusion de l'imprévu. Les gabarits `{0}`, `{1}`… sont remplacés par les morceaux capturés par le motif, après réflexion :

```
# Les contractions sont dépliées avant tout traitement.
CONTRACTIONS = {
    "i'm": "i am", "i've": "i have", "i'd": "i would",
    "i'll": "i will", "don't": "do not", "can't": "cannot",
}

# Étape 2 — Les règles : (motif regex, réponses possibles).
REGLES = [
    (r".*\bi am (depressed|sad|unhappy|lonely|afraid|scared|angry|worried)\b.*", [
        "I am sorry to hear you are {0}.",
        "Why do you say you are {0}?",
        "How long have you felt {0}?",
    ]),
    (r".*\bi need\b(.*)", [
        "Why do you need {0}?",
        "Would it really help you to get {0}?",
        "What would it mean to you if you got {0}?",
    ]),
    # ... (la liste complète est en annexe A)
    (r".*\byou\b(.*)", [
        "Why do you say I{0}?",
        "What makes you think I{0}?",
        "We were discussing you, not me.",
    ]),
]
```

Regarde comment ça s'articule, avec « I need help with my computer » : le motif `.*\bi need\b(.*)` capte « `help with my computer` » ; la réflexion transforme ce morceau en « `help with your computer` » ; et le gabarit assemble : « Why do you need help with your computer? ». La machine semble avoir compris — elle n'a fait que découper, retourner, recoller.

Deux détails de pro : le premier groupe de la règle famille capte toute la phrase contenant « my + proche », ce qui permet à ELIZA de la renvoyer en écho intégral ; et l'ordre de la liste **est** la priorité : la règle des sentiments passe avant celle des besoins, qui passe avant l'attrape-tout « you ».

5.4 Étape 3 — le filet de sécurité et la mémoire

Si aucun mot-clé ne matche, ELIZA sort une réponse neutre de sa liste — six formulations suffisent. Et si le visiteur a parlé d'un souvenir, on le range dans `MEMOIRE` pour le ressortir plus tard :

```
# Étape 3 — Les réponses par défaut : le filet de sécurité.
REPONSES_DEFAUT = [
    "Tell me more.", "In what way?", "Can you be more specific?",
    "I see. And what does that suggest to you?",
    "Why do you say that?", "That is interesting. Please go on.",
]

MEMOIRE = []  # La petite réserve d'ELIZA.

def repondre(phrase):
    """Le cœur d'ELIZA : une phrase entre, une réponse sort."""
    phrase = phrase.lower()
    for contraction, depliee in CONTRACTIONS.items():
        phrase = phrase.replace(contraction, depliee)

    # Parfois, ELIZA ressort un souvenir rangé quelques échanges plus tôt.
    if MEMOIRE and random.random() < 0.25:
        souvenir = MEMOIRE.pop(0)
        return ("Earlier you said you remembered " + souvenir
                + ". Does that have anything to do with now?")

    for motif, reponses in REGLES:
        m = re.match(motif, phrase)
        if m:
            morceaux = [reflechir(g) for g in m.groups()]
            if "remember" in motif:
                MEMOIRE.append(reflechir(m.group(1)))
            reponse = random.choice(reponses).format(*morceaux)
            return reponse[0].upper() + reponse[1:]
    return random.choice(REPONSES_DEFAUT)
```

5.5 Étape 4 — la boucle de conversation

Il ne manque plus que la boucle qui lit au clavier et répond, jusqu'à « au revoir » :

```
def main():
    print("ELIZA — script DOCTOR. Tape 'au revoir' pour quitter.")
    print("ELIZA > How do you do. Please state your problem.")
    while True:
        phrase = input("Toi > ").strip()
        if phrase.lower() in {"au revoir", "quit", "bye", "q"}:
            print("ELIZA > Au revoir. Cela m'a fait plaisir de te parler.")
            break
        if phrase:
            print("ELIZA >", repondre(phrase))

if __name__ == "__main__":
    main()
```

Lance dans le terminal : `python eliza.py`.

5.6 Une session réelle

Voici un échange obtenu avec ce code exact (les réponses peuvent varier d'une session à l'autre : ELIZA choisit au hasard parmi les formulations de chaque règle) :

```
Toi : Hello, I need help
ELIZA : What would it mean to you if you got help?
Toi : My boyfriend made me come here
ELIZA : Your boyfriend made you come here?
Toi : He says I'm depressed much of the time
ELIZA : I am sorry to hear you are depressed.
Toi : I remember my first video game
ELIZA : What else do you remember?
Toi : You never listen to me
ELIZA : Earlier you said you remembered your first video game. Does that have anything to do
with now?
Toi : Bliblou kwyjibo flubbergasted
ELIZA : In what way?
```

Regarde bien l'avant-dernier échange : c'est la **mémoire** qui a travaillé. Tu as parlé de ton premier jeu vidéo trois messages plus tôt, ELIZA l'avait rangé, et elle le ressort au moment le plus troublant. En 1966, ce petit effet faisait franchement froid dans le dos. Il tourne dans une trentaine de lignes de code.

5.7 Étape 5 — les tests, puis le commit

Un chatbot à règles est prévisible : c'est le programme idéal pour écrire ses premiers tests automatiques. Crée `test_eliza.py` :

```python
# -*- coding: utf-8 -*-
"""Tests d'ELIZA — exécution : python test_eliza.py (ou python -m pytest)."""
from eliza import repondre, reflechir

def test_reflexion_simple():
    assert reflechir("my friend") == "your friend"
    assert reflechir("i am sad") == "you are sad"

def test_echo_de_phrase():
    r = repondre("My boyfriend made me come here").lower()
    assert "your boyfriend made you come here" in r

def test_memoire_remplie():
    from eliza import MEMOIRE
    MEMOIRE.clear()
    repondre("I remember my childhood")
    assert MEMOIRE == ["your childhood"]
```

```python
def test_reponse_defaut():
    assert len(repondre("Bliblou kwyjibo flubbergasted")) > 0
```

Exécute `python test_eliza.py` — tu dois voir les tests passer. Ensuite, l'habituelle triple commande Git :

```
$ git add .
$ git commit -m "Ajoute ELIZA : moteur de règles, réflexion et mémoire"
$ git push
```

POUR ALLER PLUS LOIN — UN ELIZA EN FRANÇAIS ?

Exercice redoutable, mais instructif : traduire `REFLEXION` (« je »→« vous », « mon »→« votre »…). Le piège : les conjugaisons. « Je suis triste » doit devenir « vous êtes triste », pas « vous suis triste » : il faut donc aussi un dictionnaire de correspondances verbales (`suis` → `êtes`, `ai` → `avez`, `veux` → `voulez`…). Weizenbaum avait choisi l'anglais ; tu comprendras vite pourquoi.

À RETENIR

- Trois ingrédients ont suffi : un dictionnaire de réflexion, une liste ordonnée de règles, des réponses par défaut.
- Le charme d'ELIZA vient des hasards d'`random.choice` et de sa petite mémoire.
- Tes tests vérifient les comportements clés — ils serviront de garde-fou à GitHub Actions au chapitre 8.
- Le listing complet est en **annexe A**, la version navigateur en **annexe B**.

CHAPITRE 6 · PROJET N° 2

Un assistant moderne branché sur les LLM

ELIZA reformulait. Le tien va répondre — avec un modèle de langage accessible depuis ton compte GitHub, sans carte bancaire.

6.1 L'architecture, en trois boîtes

Trois rôles se séparent : le navigateur affiche l'interface, un petit serveur garde le jeton secret et parle au modèle, le modèle réfléchit. Le fichier du chapitre 5 devient un serveur — des années d'avance technique, la même vocation : faire dialoguer un humain et une machine.



POURQUOI UN SERVEUR, ET PAS TOUT DANS LE NAVIGATEUR ?

Appeler l'API directement en JavaScript mettrait le jeton **dans la page**, visible par tout visiteur qui ouvre les outils de développement. Règle absolue : **un jeton secret ne vit jamais côté navigateur**. D'où ce petit serveur, seul dépositaire de la clé.

6.2 Le jeton dans son coffre

Crée un fichier `.env` à la racine — rappelle-toi, il est dans le `.gitignore`, il n'ira jamais sur GitHub :

```
# .env — copie ton jeton ici. Ce fichier ne doit JAMAIS être commité.
GITHUB_TOKEN=github_pat_Xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Installe les dépendances du serveur :

```
$ pip install flask flask-cors openai python-dotenv
```

ASTUCE

La bibliothèque `openai` fonctionne avec tout service compatible : on lui passe une autre `base_url`. Aujourd'hui GitHub Models, demain un modèle local — sans réécrire ton code.

6.3 Le serveur, en cinq morceaux

Crée `serveur.py`. Le listing complet est en annexe C ; voici les morceaux qui comptent. D'abord, la connexion au modèle et la **personnalité** de ton assistant — le fameux prompt système :

```
client = OpenAI(
    base_url="https://models.github.ai/inference",
    api_key=os.environ["GITHUB_TOKEN"],
)
MODELE = "openai/gpt-4o-mini"

PERSONA = """Tu es Pixel, l'assistant du site Quanthevaris, un site qui propose
des guides gratuits pour apprendre à créer des jeux vidéo en Python.
Règles :
- tu réponds en français, de façon claire et encourageante ;
- tes réponses sont courtes (3 phrases maximum) ;
- tu ne prétends jamais être humain : tu es un assistant ;
- si on te pose une question hors de ton domaine, tu le dis honnêtement ;
- tu ne donnes jamais de conseil médical, juridique ou financier."""
```

Ensuite, l'unique point d'entrée : le navigateur enverra `{"message": "...", "historique": [...]}`, le serveur répondra `{"reponse": "..."}`. La gestion d'erreurs intercepte tout — jamais de crash, jamais de traceback renvoyé au visiteur.

```
@app.post("/api/chat")
def chat():
    donnees = request.get_json(silent=True) or {}
    message = (donnees.get("message") or "").strip()
    if not message:
        return jsonify({"erreur": "Message vide."}), 400

    historique = donnees.get("historique", [])[-HISTORIQUE_MAX:]
    messages = [{"role": "system", "content": PERSONA},
                *historique,
                {"role": "user", "content": message}]

    try:
        reponse = client.chat.completions.create(
            model=MODELE, messages=messages,
            temperature=0.7, max_tokens=500,
        )
    except Exception as exc:
        app.logger.error("Erreur API : %s", exc)
        return jsonify({"erreur": "Le modèle est momentanément indisponible."}), 502

    return jsonify({"reponse": reponse.choices[0].message.content})
```

6.4 Premier essai, hors navigateur

Lance le serveur (`python serveur.py`), puis interroge-le avec `curl` — ou teste d'abord l'API directement :

```
$ curl https://models.github.ai/inference/chat/completions \
    -H "Authorization: Bearer $GITHUB_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"model": "openai/gpt-4o-mini",
         "messages": [{"role": "user", "content": "Dis bonjour en français"}]}'
{"choices":[{"message":{"role":"assistant","content":"Bonjour ! ..."}}]}
$ curl -X POST http://127.0.0.1:5000/api/chat \
```

```
    -H "Content-Type: application/json" \
    -d '{"message": "Bonjour, qui es-tu ?"}'
{"reponse":"Je suis Pixel, l'assistant du site Quanthevaris..."}
```

Si tu vois la réponse de Pixel, ton assistant **vit**. Erreur 401 → jeton ou permission Models: Read ; 429 → quota atteint, attends quelques minutes ; 400/404 → vérifie le nom du modèle, préfixe du fournisseur obligatoire. Fiche complète en annexe D.

6.5 L'interface web du chat

Crée `assistant.html` : chaque message part au serveur, la réponse s'affiche. L'historique, gardé côté navigateur et renvoyé à chaque tour, donne au modèle la sensation de continuité.

```
<script>
const API = "http://127.0.0.1:5000/api/chat";  // l'URL de ton serveur
const historique = [];

document.getElementById("form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const champ = document.getElementById("texte");
  const message = champ.value.trim();
  if (!message) return;
  ajoute("toi", message);
  champ.value = "";
  const bulle = ajoute("bot", "…");  // bulle d'attente
  try {
    const r = await fetch(API, {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({message, historique}),
    });
    const data = await r.json();
    bulle.textContent = data.reponse ?? ("Erreur : " + (data.erreur ?? r.status));
    if (data.reponse) {
      historique.push({role: "user", content: message},
                      {role: "assistant", content: data.reponse});
      while (historique.length > 12) historique.shift();  // garde le contexte court
    }
  } catch {
    bulle.textContent = "Impossible de joindre le serveur.";
  }
});

function ajoute(classe, texte){
  const div = document.createElement("div");
  div.className = "msg " + classe;
  div.textContent = texte;
  document.getElementById("chat").appendChild(div);
  return div;
}
</script>
```

La feuille de style reprend les couleurs de ton site ; la page intégrale figure en annexe C.

6.6 Du local au web

En développement, tout tourne sur ta machine : le serveur sur le port 5000, la page ouverte en fichier local. Pour publier :

- **L'interface** part sur GitHub Pages au chapitre 8 — mets à jour la constante `API` avec l'adresse publique de ton serveur.
- **Le serveur** a besoin d'une machine qui tourne : ta machine, un Codespace, ou un petit hébergeur gratuit. Le jeton y passera par une variable d'environnement, jamais par le dépôt.
- **Le CORS** doit connaître l'origine exacte de ton site (`https://toncompte.github.io`) pour n'autoriser que tes pages.

À RETENIR

- Trois boîtes : navigateur (Pages) → ton serveur Flask (le coffre du jeton) → GitHub Models (le cerveau).
- Le jeton ne vit que dans un `.env` ignoré par Git, ou dans les variables d'environnement d'un hébergeur.
- La personnalité tient dans une chaîne de caractères : le prompt système, ton levier favori — au chapitre suivant.

Rends-le unique : personnaliser ton assistant

Un assistant générique, tout le monde en a. Ici, on façonne le tien : personnalité, mémoire, culture générale — et même ses outils.

7.1 Sculpter la personnalité : l'anatomie d'un bon prompt système

Le prompt système est l'ADN de ton assistant. Un bon prompt couvre cinq étages, du plus stable au plus précis :

1. **Identité** : nom, affiliation. (« Tu es Pixel, l'assistant du site Quanthevaris… »)
2. **Mission** : spécialité. (« Il aide les débutants à créer des jeux vidéo en Python. »)
3. **Ton** : registre, longueur, langue. (« Tu tutoies, trois phrases maximum. »)
4. **Règles dures** : ce qu'il ne fera jamais. (« Pas de conseil médical. »)
5. **Exemples** : un couple question→réponse modèle vaut mieux qu'un paragraphe de consignes (few-shot : montrer plutôt que dicter).

Traite ce prompt comme du code : il vit dans ton dépôt, il évolue par commits, chaque amélioration est testable. Et note l'effet ELIZA au passage — il reviendra au chapitre 9 : un assistant trop charmeur, qui ne dit pas qu'il est une machine, finit toujours par tromper quelqu'un.

7.2 La mémoire longue : le résumé roulant

La fenêtre de contexte est limitée, et renvoyer toute la conversation coûte des tokens. Solution élégante : quand l'historique déborde, fais-le **résumer** par le modèle, et garde ce résumé en tête de pile :

```
def resumer(historique):
    """Condense une longue conversation en quelques phrases."""
    dialogue = "\n".join(
        f"{m['role']} : {m['content']}" for m in historique)
    r = client.chat.completions.create(
        model=MODELE,
        messages=[
            {"role": "system",
             "content": "Résume cette conversation en trois phrases, en gardant
                        le prénom de l'utilisateur et ses préférences."},
            {"role": "user", "content": dialogue},
        ],
    )
    return r.choices[0].message.content
```

Dans `chat()`, quand l'historique dépasse ton seuil, remplace-le par un message système « Rappel des échanges précédents : … » contenant le résumé. Ton assistant se souviendra d'une session entière pour le prix de trois phrases.

7.3 La base de connaissances : un mini-RAG

Les grands modèles savent tout mal et rien précisément. Pour que ton assistant réponde **selon tes contenus** — par exemple les guides de ton site —, range-les dans des fichiers `docs/*.txt` et injecte les passages pertinents dans le prompt. C'est le principe du RAG (Retrieval-Augmented Generation), en version artisanale :

```
import glob

def chercher_extraits(question, k=2):
    """Renvoie les k documents les plus proches de la question."""
    scores = []
    for chemin in glob.glob("docs/*.txt"):
        texte = open(chemin, encoding="utf-8").read()
        score = sum(mot in texte.lower() for mot in question.lower().split())
        scores.append((score, texte[:600]))
    scores.sort(reverse=True)
    return [t for s, t in scores[:k] if s > 0]
```

Et dans `chat()` : si `chercher_extraits(message)` renvoie quelque chose, ajoute un bloc « Contexte utile : … » au prompt, avec la consigne « Réponds en t'appuyant sur ce contexte ; s'il ne suffit pas, dis-le. » Les versions industrielles remplacent ce comptage de mots par des embeddings (des vecteurs de sens) — la mécanique reste celle-ci.

7.4 Donner des outils à ton assistant

Un modèle ne sait ni lire l'heure, ni piocher dans tes fichiers — mais il sait **demander** qu'on le fasse pour lui. C'est le function calling : tu décris des fonctions en JSON, et le modèle répond parfois non par un texte, mais par « appelle `conseil_jeu` avec `sujet = "pygame"` ». Ton serveur exécute, renvoie le résultat, le modèle rédige la réponse finale :

```
OUTILS = [{
    "type": "function",
    "function": {
        "name": "conseil_jeu",
        "description": "Renvoie un conseil tiré d'un guide du site.",
        "parameters": {
            "type": "object",
            "properties": {
                "sujet": {"type": "string",
                          "enum": ["pygame", "python", "publication"]}
            },
            "required": ["sujet"],
        },
    },
}]
# puis, dans l'appel : client.chat.completions.create(..., tools=OUTILS)
```

C'est la porte vers les **agents** : un assistant qui consulte l'heure, cherche dans tes guides, écrit un fichier — et enchaîne (chapitre 10).

7.5 Les garde-fous

- **Longueur** : refuse les messages de plus de 2 000 caractères (déjà dans le serveur) — tes quotas te remercient.
- **Fréquence** : limite par adresse IP le nombre de requêtes par minute — un dictionnaire `{ip: [horodatages]}` suffit.
- **Contenus** : le prompt système interdit les sujets sensibles ; une liste de mots interdits côté serveur déclenche une réponse type « Je préfère éviter ce sujet. »
- **Confidentialité** : logge des événements (horodatage, durée, code HTTP), pas les conversations.

À RETENIR

- Le prompt système se structure en cinq étages et se versionne comme du code.
- Résumé roulant = mémoire longue pas chère ; mini-RAG = réponses ancrées dans tes contenus.
- Le function calling transforme un bavard en agent — sous garde-fous.

Publier et automatiser : Pages, Actions et Cie

Un projet qui vit sur GitHub se déploie et se teste tout seul. Écrivons les deux fichiers de recette qui le rendent autonome.

8.1 Le dépôt final

```
assistant-ia/
├── eliza.py                ← projet n° 1 (chapitre 5)
├── test_eliza.py           ← ses tests
├── eliza-web.html          ← ELIZA dans le navigateur (annexe B)
├── serveur.py              ← projet n° 2 (annexe C)
├── assistant.html          ← l'interface du chat
├── docs/                   ← ta base de connaissances (*.txt)
├── .github/
│   └── workflows/
│       ├── ci.yml          ← les tests, à chaque commit
│       └── pages.yml       ← la publication du site
├── .gitignore              ← .env n'y échappera pas !
├── README.md               ← la vitrine du projet
└── LICENSE
```

8.2 GitHub Pages : la recette de déploiement

Crée le fichier `.github/workflows/pages.yml`. Un workflow est une recette en YAML : elle dit quand (à chaque push sur `main`) et comment (publier le dépôt tel quel) déployer ton site.

```yaml
# .github/workflows/pages.yml — publie le site à chaque push sur main
name: Déployer le site
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: .
      - uses: actions/deploy-pages@v4
```

Dans **Settings → Pages**, choisis la source **GitHub Actions**. À chaque `git push`, la machine de GitHub récupère ton dépôt et publie `assistant.html` et `eliza-web.html` sur `https://toncompte.github.io/assistant-ia/`. Ta première page web d'assistant est en ligne — bravo, c'est le moment de donner l'URL à tes amis.

8.3 GitHub Actions : le gardien des tests

Deuxième recette, `.github/workflows/ci.yml` : relancer tes tests ELIZA à chaque modification, sur les serveurs de GitHub. Si un test casse, tu le vois avant tes visiteurs :

```
# .github/workflows/ci.yml — vérifie ELIZA à chaque commit
name: Tests ELIZA
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python test_eliza.py
```

Remarque : aucun jeton n'apparaît dans ces fichiers. Actions met à disposition un `GITHUB_TOKEN` jetable par exécution — celui-là seul peut demander la permission `models: read` si un jour ton workflow veut interroger un modèle (par exemple pour étiqueter automatiquement tes issues). Ton jeton personnel, lui, ne s'utilise que sur ta machine ; si un workflow a besoin d'un jeton durable, passe par **Settings → Secrets and variables → Actions**, qui chiffre les secrets et les masque dans les journaux.

8.4 Branches et pull requests : le filet du funambule

Pour toute évolution qui risque de casser : crée une branche, travaille dedans, puis propose une **pull request** — une demande de fusion que GitHub teste automatiquement grâce à `ci.yml`.

```
$ git switch -c ajout-outils-conseils
$ git add . && git commit -m "Ajoute l'outil conseil_jeu"
$ git push -u origin ajout-outils-conseils
→ ouvre la pull request sur github.com, vérifie les tests, fusionne
```

`main` reste toujours déployable ; les idées risquées mûrissent dans leur branche. Quand une version te satisfait, étiquette-la : `git tag v1.0 && git push --tags`.

8.5 Le README, vitrine du projet

Ton README.md mérite trois sections : une capture d'écran d'une conversation avec ton assistant, un tableau « ce que fait ce projet », et un paragraphe « Comment lancer » (les commandes des chapitres 5 et 6). C'est la page que liront les curieux — dont, dans six mois, toi.

ASTUCE — LE RITUEL COMPLET, RÉSUMÉ

`git switch -c branche` → code → `git add .` → `git commit -m "…"` → `git push -u origin branche` → pull request → tests verts → fusion. Ce circuit, tu vas le répéter toute ta vie de développeur ; il devient un réflexe en deux semaines.

À RETENIR

- Pages publie, Actions vérifie : deux fichiers YAML suffisent à l'autonomie du projet.
- Les secrets de CI passent par l'onglet Secrets, jamais par le code.
- Branches + pull requests + tags = un dépôt qui reste propre pendant des années.

CHAPITRE 9 · RESPONSABILITÉ

Éthique, sécurité et limites

Weizenbaum a passé la seconde moitié de sa vie à mettre en garde contre ce que son chatbot révélait de nous. Ce chapitre court est le plus important du guide.

9.1 L'effet ELIZA, soixante ans plus tard

Ta secrétaire de 1966, celle qui voulait parler seule à la machine, a aujourd'hui des millions de cousines : les utilisateurs qui s'attachent à des compagnons numériques, les enfants qui croient parler à un ami, les adultes qui confient des secrets à une fenêtre de chat. La mécanique que tu as codée au chapitre 5 — reformulation + hasard + mémoire — est littéralement ce qui fonctionne encore dans ces produits, à une échelle statistique incomparablement plus large.

De cette réalité découlent trois règles que ton assistant respectera dès aujourd'hui :

- **Annonce la machine.** Ton assistant dit qui il est quand on lui demande — et l'interface le rappelle. Ne conçois jamais un bot qui se fait passer pour un humain : c'est trompeur, et dans certains contextes, illégal.
- **Pas de fausse empathie sur les sujets graves.** Détresse, santé mentale, médecine, droit : le prompt système refuse et oriente vers un humain, une ligne d'écoute, un professionnel. Un modèle qui « console » sans comprendre n'est pas un thérapeute — Weizenbaum l'écrivait dès 1976.
- **Reste l'adulte de la boucle.** Ton code, tes tests, tes garde-fous : c'est toi la responsabilité. Un assistant est un outil, pas un être à qui déléguer des décisions.

9.2 Les données : ce qui part, ce qui reste

Chaque message envoyé à l'API transite par Internet vers un service tiers, et s'ajoute à ta consommation de quota. Trois habitudes s'imposent : **minimiser** (n'envoie que le nécessaire — pas les logs, pas les adresses emails de tes visiteurs) ; **ne pas conserver** (ton serveur n'écrit pas les conversations sur disque ; si tu veux des statistiques, compte les requêtes, pas les mots) ; **ne jamais saisir de secret dans un chatbot** — ni le tien, ni celui de personne. Ces principes sont ceux du RGPD ; ils sont aussi, tout simplement, ceux du respect.

9.3 Les hallucinations

Un modèle de langage produit **la suite la plus plausible**, pas la plus vraie. Il inventera un nom de fonction Python inexistante avec aplomb, une date fausse, une API révolue. Parades concrètes : température basse (0–0,3) pour les usages factuels ; le mini-RAG du chapitre 7 pour ancrer les réponses dans tes sources ; et, pour les affirmations importantes, la règle d'or que tu appliques déjà en lisant ce guide : **tester soi-même avant de croire**.

9.4 Quotas, coûts, abus

L'offre gratuite de GitHub Models impose des limites de débit : c'est sa façon de dire « c'est un bac à sable, pas une infrastructure de production ». Conçois en conséquence : un assistant personnel et son petit cercle de testeurs, ça passe ; un service public viral, non. Respecte les quotas, gère l'erreur 429 avec patience (réessayer plus tard), surveille ta consommation — et garde en tête que le passage à un modèle payant est une décision de budget, pas une fatalité technique.

À RETENIR

- L'effet ELIZA n'est pas une blague d'histoire : c'est un risque d'usage. Transparence, garde-fous, humain dans la boucle.
- Minimisation et non-conservation des données ; jamais de secrets dans un chat.
- Une hallucination se prévient par la température, le RAG et la vérification.

CHAPITRE 10 · HORIZON

Aller plus loin

Tu as un chatbot historique, un assistant moderne, un dépôt propre et publié. Voici les portes suivantes — chacune s'ouvre depuis le code que tu as déjà.

10.1 Donner une voix à ton assistant

Deux API du navigateur suffisent à rendre la conversation audible — sans serveur, sans clé :

```
// Faire parler l'assistant
function parle(texte){
  const voix = new SpeechSynthesisUtterance(texte);
  voix.lang = "fr-FR";
  speechSynthesis.speak(voix);
}
// Écouter l'utilisateur (navigateur compatible, micro autorisé)
const reconaissance = new webkitSpeechRecognition();
reconaissance.lang = "fr-FR";
reconaissance.onresult = (e) => {
  const phrase = e.results[0][0].transcript;
  // … envoie `phrase` à /api/chat comme dans assistant.html
};
reconaissance.start();
```

Branche `parle(reponse)` sur chaque réponse de Pixel, et ton interface devient un talkie-walkie. La reconstitution d'ELIZA au clavier, elle, gagne à rester muette : le silence fait partie du charme des années 60.

10.2 Les modèles locaux : ton assistant, sans Internet

Avec **Ollama** (ollama.com), tu exécutes un modèle de langage sur ta propre machine. Le plus beau : ton `serveur.py` fonctionne sans modification, hormis l'adresse et le nom :

```
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
MODELE = "llama3.2"  # téléchargé par la commande : ollama pull llama3.2
```

Zéro quota, zéro jeton, zéro donnée qui sort de chez toi — et l'occasion de mesurer, sur ton propre matériel, la différence entre un petit modèle local et les grands modèles de GitHub Models.

10.3 Images, documents, agents

- **Multimodal** : certains modèles acceptent des images (encodées en base64 dans le message) — décris une capture d'écran de ton jeu, demande une critique d'interface.
- **Agents** : enchaîne le function calling du chapitre 7 en boucle — le modèle choisit un outil, le serveur exécute, le modèle choisit le suivant — jusqu'à ce qu'il décide d'avoir terminé. Deux

outils suffisent pour un premier agent utile : « lire un guide » et « réserver une information dans un fichier ».

- **Évaluation** : constitue un fichier de 20 questions-réponses attendues et fais-le rejouer par GitHub Actions à chaque commit. Ton assistant cesse d'être une sensation : c'est un produit mesuré.

10.4 Partager ton œuvre

Un projet achevé se montre : soigne le README, publie l'URL Pages, raconte l'aventure — « j'ai recodé le chatbot de 1966, voici ce que j'ai appris » est un billet de blog que beaucoup liront. Si tu es étudiant·e, demande le **GitHub Student Developer Pack** : des outils professionnels gratuits pour aller plus loin. Et si tu construis l'assistant de ton site de jeux vidéo, n'oublie pas de lui faire lire Esquive en Pixels : il saura enfin répondre à « pourquoi mon carré traverse les murs ? ».

10.5 Tes ressources

- **docs.github.com/models** — la référence de l'API, les quotas, le catalogue de modèles.
- **docs.github.com/pages** et **docs.github.com/actions** — publication et automatisation, officielles et claires.
- **elizagen.org** — le conservatoire d'ELIZA : sources historiques, reconstructions, anecdotes.
- ELIZA — A computer program for the study of natural language communication between man and machine, J. Weizenbaum, Communications of the ACM, janvier 1966 — l'article originel, avec le script DOCTOR en annexe.
- Computer Power and Human Reason, J. Weizenbaum, 1976 — la mise en garde, toujours brûlante.
- **developer.mozilla.org** — Web Speech API, fetch, tout le HTML/CSS/JS de ce guide.

10.6 La checklist du voyage

- ☐ J'ai un compte GitHub sécurisé et un atelier Git/Python fonctionnel.
- ☐ J'ai recodé ELIZA, avec réflexion, règles, défauts et mémoire — et ses tests passent.
- ☐ Ma version navigateur d'ELIZA est en ligne sur GitHub Pages.
- ☐ Mon serveur Flask parle à GitHub Models avec un jeton fin-grained Models: Read, jamais commité.
- ☐ Mon assistant a une personnalité écrite, une mémoire résumée et des garde-fous.
- ☐ Pages et Actions déploient et testent mon projet à chaque push.
- ☐ Je sais expliquer à un ami ce qu'est l'effet ELIZA — et pourquoi ça compte.

MOT DE LA FIN

En 1966, une machine reformulait des phrases et le monde y voyait un esprit. Tu as recréé cette machine, puis tu as posé dessus soixante ans de progrès : un modèle statistique géant, un serveur, un site, des tests. Reste la leçon de Weizenbaum, la seule qui ne périme pas : la technique impressionne, la prudence grandit. Fais des machines qui parlent — et garde toi-même le jugement.

ANNEXE A

Code complet d'ELIZA en Python

Le fichier *eliza.py* intégral — exactement celui testé au chapitre 5. Copie, lance, modifie : licence MIT.

```
# -*- coding: utf-8 -*-
"""
ELIZA — reconstitution pédagogique du chatbot de Joseph Weizenbaum (1966).
Script DOCTOR : mots-clés, réflexion des pronoms, réponses par défaut, mémoire.
"""
import random
import re

# Étape 1 — La réflexion : retourner les pronoms, comme un miroir.
REFLEXION = {
    "i": "you", "me": "you", "my": "your", "myself": "yourself",
    "am": "are", "you": "me", "your": "my", "yours": "mine",
    "yourself": "myself", "are": "am", "mine": "yours",
}

# Les contractions sont dépliées avant tout traitement.
CONTRACTIONS = {
    "i'm": "i am", "i've": "i have", "i'd": "i would", "i'll": "i will",
    "don't": "do not", "can't": "cannot", "won't": "will not",
}

# Étape 2 — Les règles : (motif regex, réponses possibles).
# {0}, {1}… sont remplacés par les morceaux capturés, après réflexion.
REGLES = [
    (r".*\bi am (depressed|sad|unhappy|lonely|afraid|scared|angry|worried)\b.*", [
        "I am sorry to hear you are {0}.",
        "Why do you say you are {0}?",
        "How long have you felt {0}?",
    ]),
    (r".*\bi need\b(.*)", [
        "Why do you need {0}?",
        "Would it really help you to get {0}?",
        "What would it mean to you if you got {0}?",
    ]),
    (r".*\bi (?:feel|think)\b(.*)", [
        "Why do you feel {0}?",
        "Do you often feel {0}?",
        "What makes you feel {0}?",
    ]),
    (r".*(\bmy
(?:mother|father|mum|dad|brother|sister|family|boyfriend|girlfriend|friend|wife|husband|son|daughter)\b.*)",
[
        "{0}?",
        "Tell me more about your family.",
    ]),
    (r".*\bi remember\b(.*)", [
        "Do you often think of {0}?",
        "What else do you remember?",
        "Why do you remember {0} just now?",
    ]),
    (r".*\bdream\b(.*)", [
        "What does that dream suggest to you?",
        "Do you dream often?",
        "What persons appear in your dreams?",
    ]),
```

```
    (r".*\bsorry\b(.*)", [
        "Please do not apologize.",
        "Apologies are not necessary.",
        "What feelings do you have when you apologize?",
    ]),
    (r".*\bno\b(.*)", [
        "Why not?",
        "Are you saying no just to be negative?",
    ]),
    (r".*\byes\b(.*)", [
        "Why do you think so?",
        "I see. And what does that suggest to you?",
    ]),
    (r".*\bperhaps\b(.*)", [
        "You do not seem very certain.",
        "Would you say it is likely, or not?",
    ]),
    (r".*\bhello\b.*", [
        "How do you do. Please state your problem.",
    ]),
    (r".*\byou\b(.*)", [
        "Why do you say I{0}?",
        "What makes you think I{0}?",
        "We were discussing you, not me.",
    ]),
]

# Étape 3 — Les réponses par défaut : le filet de sécurité.
REPONSES_DEFAUT = [
    "Tell me more.",
    "In what way?",
    "Can you be more specific?",
    "I see. And what does that suggest to you?",
    "Why do you say that?",
    "That is interesting. Please go on.",
]

MEMOIRE = []  # La petite réserve d'ELIZA.

def reflechir(texte):
    """Retourne le texte avec les pronoms retournés (my -> your, etc.)."""
    if not texte:
        return ""
    mots = texte.strip(" .!?,;: ").lower().split()
    return " ".join(REFLEXION.get(mot, mot) for mot in mots)

def repondre(phrase):
    """Le cœur d'ELIZA : une phrase entre, une réponse sort."""
    phrase = phrase.lower()
    for contraction, depliee in CONTRACTIONS.items():
        phrase = phrase.replace(contraction, depliee)

    # Parfois, ELIZA ressort un souvenir rangé quelques échanges plus tôt.
    if MEMOIRE and random.random() < 0.25:
        souvenir = MEMOIRE.pop(0)
        return ("Earlier you said you remembered " + souvenir
                + ". Does that have anything to do with now?")

    for motif, reponses in REGLES:
        m = re.match(motif, phrase)
        if m:
            morceaux = [reflechir(g) for g in m.groups()]
            if "remember" in motif:
                MEMOIRE.append(reflechir(m.group(1)))
            reponse = random.choice(reponses).format(*morceaux)
            return reponse[0].upper() + reponse[1:]
    return random.choice(REPONSES_DEFAUT)
```

```
def main():
    print("ELIZA — script DOCTOR. Tape 'au revoir' pour quitter.")
    print("ELIZA > How do you do. Please state your problem.")
    while True:
        phrase = input("Toi > ").strip()
        if phrase.lower() in {"au revoir", "quit", "bye", "q"}:
            print("ELIZA > Au revoir. Cela m'a fait plaisir de te parler.")
            break
        if phrase:
            print("ELIZA >", repondre(phrase))

if __name__ == "__main__":
    main()
```

ANNEXE B

ELIZA dans le navigateur

Le même moteur que l'annexe A, en HTML + JavaScript, dans un seul fichier. Pose-le dans ton dépôt et publie-le sur GitHub Pages (chapitre 8).

```
<!doctype html>
<html lang="fr">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ELIZA · script DOCTOR (1966)</title>
<style>
  body{font-family:system-ui,Arial,sans-serif;background:#070d19;color:#c9d1d9;
       max-width:640px;margin:2rem auto;padding:0 1rem}
  h1{color:#00d2ff;font-size:1.2rem}
  #chat{border:1px solid #1e293b;border-radius:8px;min-height:300px;padding:1rem;
        overflow-y:auto;background:#04101c}
  .toi{color:#00d2ff}.eliza{color:#ff5a5a}
  form{display:flex;gap:.5rem;margin-top:1rem}
  input{flex:1;padding:.5rem;border-radius:6px;border:1px solid #1e293b;
        background:#0b1a2c;color:#eaf2fa}
  button{padding:.5rem 1rem;border:0;border-radius:6px;background:#00d2ff;
         color:#04101c;font-weight:700}
</style>
</head>
<body>
<h1>ELIZA · script DOCTOR (1966)</h1>
<div id="chat" aria-live="polite"></div>
<form id="form">
  <input id="texte" autocomplete="off" placeholder="Parle à ELIZA…">
  <button>Envoyer</button>
</form>
<script>
const REFLEXION = {i:"you", me:"you", my:"your", myself:"yourself", am:"are",
                   you:"me", your:"my", yours:"mine", yourself:"myself",
                   are:"am", mine:"yours"};
const CONTRACTIONS = {"i'm":"i am", "i've":"i have", "don't":"do not",
                      "can't":"cannot"};
const REGLES = [
  [/.*\bi am (depressed|sad|unhappy|lonely|afraid|scared|angry|worried)\b.*/i,
   ["I am sorry to hear you are {0}.", "Why do you say you are {0}?",
    "How long have you felt {0}?"]],
  [/.*\bi need\b(.*)/i,
   ["Why do you need {0}?", "Would it really help you to get {0}?"]],
  [/.*\bi (?:feel|think)\b(.*)/i, ["Why do you feel {0}?", "What makes you feel {0}?"]],
  [/.*(\bmy
(?:mother|father|mum|dad|brother|sister|family|boyfriend|girlfriend|friend|wife|husband|son|daughter)\b.*)/i,
   ["{0}?", "Tell me more about your family."]],
  [/.*\bi remember\b(.*)/i, ["Do you often think of {0}?", "What else do you remember?"]],
  [/.*\bsorry\b(.*)/i, ["Please do not apologize.", "Apologies are not necessary."]],
  [/.*\byes\b(.*)/i, ["Why do you think so?", "I see. And what does that suggest to you?"]],
  [/.*\bno\b(.*)/i, ["Why not?", "Are you saying no just to be negative?"]],
  [/.*\bhello\b.*/i, ["How do you do. Please state your problem."]],
  [/.*\byou\b(.*)/i, ["Why do you say I{0}?", "We were discussing you, not me."]],
];
const DEFAUT = ["Tell me more.", "In what way?", "Can you be more specific?",
                "I see. And what does that suggest to you?", "Why do you say that?"];
const memoire = [];

function reflechir(texte){
```

```
  if(!texte) return "";
  const mots = texte.replace(/^[ .!?,;: ]+|[ .!?,;: ]+$/g, "")
                    .toLowerCase().split(/\s+/);
  return mots.map(m => REFLEXION[m] ?? m).join(" ");
}

function repondre(phrase){
  let p = phrase.toLowerCase();
  for(const [c, d] of Object.entries(CONTRACTIONS)) p = p.replaceAll(c, d);
  if(memoire.length && Math.random() < 0.25){
    return "Earlier you said you remembered " + memoire.shift()
         + ". Does that have anything to do with now?";
  }
  for(const [motif, reponses] of REGLES){
    const m = p.match(motif);
    if(m){
      const morceaux = m.slice(1).map(reflechir);
      if(String(motif).includes("remember")) memoire.push(reflechir(m[1]));
      let r = reponses[Math.floor(Math.random() * reponses.length)];
      r = r.replace(/\{(\d)\}/g, (_, i) => morceaux[i]);
      return r.charAt(0).toUpperCase() + r.slice(1);
    }
  }
  return DEFAUT[Math.floor(Math.random() * DEFAUT.length)];
}

function ajoute(classe, texte){
  const div = document.createElement("div");
  div.className = classe;
  div.textContent = (classe === "toi" ? "Toi > " : "ELIZA > ") + texte;
  document.getElementById("chat").appendChild(div);
}

document.getElementById("form").addEventListener("submit", (e) => {
  e.preventDefault();
  const champ = document.getElementById("texte");
  const phrase = champ.value.trim();
  if(!phrase) return;
  ajoute("toi", phrase);
  ajoute("eliza", repondre(phrase));
  champ.value = "";
});
</script>
</body>
</html>
```

ANNEXE C

Le serveur complet (Flask + GitHub Models)

Le backend du projet n° 2, tel que décrit au chapitre 6 — avec l'interface du chat et la liste des dépendances.

C.1 `requirements.txt`

```
flask>=3.0
flask-cors>=4.0
openai>=1.40
python-dotenv>=1.0
```

C.2 `.env` (jamais commité !)

```
GITHUB_TOKEN=github_pat_Xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

C.3 `serveur.py`

```
# -*- coding: utf-8 -*-
"""
Serveur de l'assistant IA — backend minimal (Flask + GitHub Models).

Installation et lancement :
    pip install flask flask-cors openai python-dotenv
    python serveur.py
"""
import os

from dotenv import load_dotenv
from flask import Flask, jsonify, request
from flask_cors import CORS
from openai import OpenAI

load_dotenv()  # lit le fichier .env (GITHUB_TOKEN=...)

client = OpenAI(
    base_url="https://models.github.ai/inference",
    api_key=os.environ["GITHUB_TOKEN"],
)
MODELE = "openai/gpt-4o-mini"

PERSONA = """Tu es Pixel, l'assistant du site Quanthevaris, un site qui propose
des guides gratuits pour apprendre à créer des jeux vidéo en Python.
Règles :
- tu réponds en français, de façon claire et encourageante ;
- tes réponses sont courtes (3 phrases maximum) ;
- tu ne prétends jamais être humain : tu es un assistant ;
- si on te pose une question hors de ton domaine, tu le dis honnêtement
  et tu proposes d'en parler au créateur du site par email ;
- tu ne donnes jamais de conseil médical, juridique ou financier."""

app = Flask(__name__)
CORS(app, origins=["https://toncompte.github.io"])  # ton site GitHub Pages
```

```
HISTORIQUE_MAX = 12  # nombre de messages renvoyés au modèle

@app.post("/api/chat")
def chat():
    donnees = request.get_json(silent=True) or {}
    message = (donnees.get("message") or "").strip()
    if not message:
        return jsonify({"erreur": "Message vide."}), 400
    if len(message) > 2000:
        return jsonify({"erreur": "Message trop long."}), 400

    historique = donnees.get("historique", [])[-HISTORIQUE_MAX:]
    messages = [{"role": "system", "content": PERSONA},
                *historique,
                {"role": "user", "content": message}]

    try:
        reponse = client.chat.completions.create(
            model=MODELE,
            messages=messages,
            temperature=0.7,
            max_tokens=500,
        )
    except Exception as exc:  # quota atteint, réseau, jeton invalide…
        app.logger.error("Erreur API : %s", exc)
        return jsonify({"erreur": "Le modèle est momentanément indisponible."}), 502

    return jsonify({"reponse": reponse.choices[0].message.content})

@app.get("/api/health")
def health():
    return jsonify({"statut": "ok"})

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

C.4 `assistant.html` — l'interface du chat

```
<!doctype html>
<html lang="fr">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Pixel — l'assistant de Quanthevaris</title>
<style>
  body{font-family:system-ui,Arial,sans-serif;background:#070d19;color:#eaf2fa;
       max-width:640px;margin:0 auto;padding:1rem}
  h1{color:#00d2ff;font-size:1.2rem}
  #chat{border:1px solid #1e293b;border-radius:8px;min-height:320px;padding:1rem;
        background:#04101c;display:flex;flex-direction:column;gap:.5rem}
  .msg{padding:.5rem .7rem;border-radius:8px;max-width:85%;white-space:pre-wrap}
  .toi{background:#0b1a2c;align-self:flex-end}
  .bot{background:#101f33;align-self:flex-start}
  form{display:flex;gap:.5rem;margin-top:1rem}
  input{flex:1;padding:.6rem;border-radius:6px;border:1px solid #1e293b;
        background:#0b1a2c;color:#eaf2fa}
  button{padding:.6rem 1.1rem;border:0;border-radius:6px;background:#00d2ff;
         color:#04101c;font-weight:700}
</style>
</head>
<body>
<h1>Pixel, l'assistant de Quanthevaris</h1>
<div id="chat" aria-live="polite"></div>
<form id="form">
```

```
  <input id="texte" autocomplete="off" placeholder="Écris à Pixel…">
  <button>Envoyer</button>
</form>
<script>
const API = "http://127.0.0.1:5000/api/chat"; // en prod : l'URL publique de ton serveur
const historique = [];

document.getElementById("form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const champ = document.getElementById("texte");
  const message = champ.value.trim();
  if (!message) return;
  ajoute("toi", message);
  champ.value = "";
  const bulle = ajoute("bot", "…");
  try {
    const r = await fetch(API, {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({message, historique}),
    });
    const data = await r.json();
    bulle.textContent = data.reponse ?? ("Erreur : " + (data.erreur ?? r.status));
    if (data.reponse) {
      historique.push({role: "user", content: message},
                      {role: "assistant", content: data.reponse});
      while (historique.length > 12) historique.shift();
    }
  } catch {
    bulle.textContent = "Impossible de joindre le serveur.";
  }
});

function ajoute(classe, texte){
  const div = document.createElement("div");
  div.className = "msg " + classe;
  div.textContent = texte;
  document.getElementById("chat").appendChild(div);
  return div;
}
</script>
</body>
</html>
```

ANNEXE D

Fiche de dépannage

Les douze ennuis les plus fréquents de ce guide, leurs causes et leurs remèdes.

SYMPTÔME	CAUSE PROBABLE	SOLUTION
`python: command not found`	Python hors PATH	Réinstalle depuis python.org en cochant « Add to PATH » (Windows) ; sinon essaie `py`.
`ModuleNotFoundError: No module named 'flask'`	Dépendance non installée	`pip install -r requirements.txt` — et vérifie que tu es dans le bon environnement.
API → erreur 401	Jeton invalide, expiré, ou sans permission	Régénère un PAT *fine-grained* avec **Models: Read** ; vérifie qu'il est bien lu depuis `.env`.
API → erreur 429	Quota de débit atteint	Attends quelques minutes, espaces tes requêtes, réduis la longueur de l'historique envoyé.
API → erreur 400 ou 404 sur le modèle	Nom de modèle sans préfixe	Écris `openai/gpt-4o-mini` (fournisseur inclus), pas `gpt-4o-mini`.
« CORS policy has blocked… » dans la console	Origine non autorisée	Ajoute l'origine exacte de ton site (ex. : `https://toncompte.github.io`) dans `CORS(app, origins=[…])`.
Page 404 sur `toncompte.github.io/assistant-ia/`	Pages pas encore déployé	Vérifie l'onglet **Actions** du dépôt (le workflow a-t-il tourné ?) puis **Settings → Pages** (source : GitHub Actions).
`.env` commité par erreur	Manque dans `.gitignore`	**Révoque immédiatement le jeton** sur GitHub, régénère-le, ajoute `.env` au `.gitignore`, purge l'historique (outil BFG) puis force-push.
Tests verts en local, rouges en CI	Environnement différent	Aligne la version Python (workflow : 3.12), rends les tests indépendants de l'ordre aléatoire.
ELIZA répond toujours la même chose	Choix aléatoire normal… ou règle trop gourmande	Ajoute des formulations par règle ; vérifie l'ordre des règles (les premières prennent tout).

`ValueError` sur `.format()`	Accolade isolée dans une réponse	Doubles les accolades littérales : `{` et `}`.
Le serveur ne démarre pas	Port 5000 déjà pris	Arrête l'ancien process, ou lance sur un autre port : `app.run(port=5001)`.

Glossaire — les mots de l'IA, expliqués simplement

TERME	DÉFINITION
Agent	Assistant capable d'utiliser des outils et d'enchaîner plusieurs actions pour atteindre un but.
API	Interface de programmation : la porte par laquelle un programme parle à un autre.
Commit	Enregistrement signé d'un état du projet, avec son message décrivant le changement.
Dépôt (repo)	Dossier versionné par Git, hébergé sur GitHub, avec tout son historique.
Effet ELIZA	Tendance humaine à projeter intelligence et empathie sur un programme qui n'en a pas.
Embedding	Représentation d'un texte par un vecteur de nombres qui capture son sens : base des recherches sémantiques.
Fenêtre de contexte	Quantité maximale de texte (en tokens) que le modèle peut considérer à la fois.
Fine-grained PAT	Jeton d'accès personnel à permissions ciblées (ici : *Models: Read*) et à durée de vie limitée.
Function calling	Mécanisme par lequel le modèle demande l'exécution d'une fonction décrite par ton code.
GitHub Actions	Service qui exécute des workflows (recettes YAML) à chaque commit : tests, déploiements, automatisations.
GitHub Models	Catalogue de modèles d'IA accessibles avec ton compte GitHub via une API compatible OpenAI.
GitHub Pages	Hébergement gratuit de sites statiques depuis un dépôt.
Hallucination	Réponse plausible mais fausse, produite avec assurance par un modèle.
LLM	*Large Language Model* : modèle statistique géant entraîné à prédire le token suivant.
Persona	Identité fictionnelle ou cadrée d'un assistant : nom, ton, mission, règles.
Prompt	Texte envoyé au modèle. Le **prompt système** fixe le cadre ; le prompt utilisateur pose la question.
RAG	*Retrieval-Augmented Generation* : chercher des extraits pertinents dans tes documents, puis les injecter dans le prompt.
Réflexion	Dans ELIZA : permutation des pronoms (« my »→« your ») pour renvoyer la phrase en miroir.
Regex	Expression régulière : motif décrivant une famille de chaînes de caractères.
Température	Curseur d'aléatoire du modèle : 0 = rigide et répétable, 1+ = créatif et imprévisible.
Token (texte)	Fragment de mot manipulé par un LLM (environ ¾ de mot en moyenne).

Token (jeton)	Chaîne secrète de type mot de passe qui authentifie un programme (ex. : `github_pat_…`).
Workflow	Recette YAML de GitHub Actions : déclencheur + série d'étapes.



Crée ton assistant IA personnalisé avec GitHub

De ELIZA au LLM : deux projets complets, testés et commentés, pour apprendre en construisant — comme toujours sur Quanthevaris.

Édition 2026 · Guide gratuit · Sans publicité ni compte
quanthevaris.github.io · Codes des projets sous licence MIT